

Trek Together

A Geospatial Trek Route Planner, Difficulty Grader & Community Trek Log

Applied Software Project Report

\WOOLF/

By

Aravind Jaimon

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

July, 2026



Scaler Mentee Email ID : aravindjaimon.dev@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 30/07/2026

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 15 May 2026 to 10 July 2026, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Aravind Jaimon

(Signature of the Candidate)

Date: 30/07/2026

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my thesis supervisor, Naman Bhalla, and to the instructors and mentors at Scaler Neovarsity, whose structured curriculum and guidance across backend engineering, databases, and system design shaped both this project and my growth as an engineer. I am equally grateful to my family for their patience and constant encouragement throughout the programme, and to my friends in the Trek Together hiking community — the group whose weekend treks inspired this application's name and its purpose. Planning routes with them surfaced every real-world question this project tries to answer: how far, how steep, how long, and how hard, really. Finally, I acknowledge the open-data and open-source communities behind OpenStreetMap, OpenTopoData, and the TypeScript ecosystem, without whose freely available data and tools an individual capstone of this scope would not have been possible.

Table of Contents

Applied Software Project.....	9
Abstract.....	9
Project Description.....	11
1.1 What the project is.....	11
1.2 Objectives.....	12
1.3 Relevance and real-life application.....	13
1.4 Development process.....	13
1.5 System overview.....	14
Requirement Gathering.....	16
2.1 Users and personas.....	16
2.2 Use cases.....	16
2.3 Functional requirements.....	18
2.4 Non-functional requirements.....	19
2.5 Feature set.....	20
Class Diagrams.....	22
3.1 The layering.....	22
3.2 Modules in the diagram.....	23
3.3 API surface.....	24
3.4 Cross-cutting design decisions.....	25
Database Schema Design.....	27
4.1 Document model overview.....	27
4.2 Schema (textual).....	28
4.3 Embed vs. reference — the design decisions.....	29
4.4 Indexes — including the two Prisma cannot express.....	30
Feature Development Process.....	32
5.1 The request flow.....	32
5.2 The elevation cache — design.....	35
5.3 Headline benchmark — cold vs. warm.....	36
5.4 Secondary benchmark — the 2dsphere index.....	37
5.5 Verification.....	37
Deployment Flow.....	40
6.1 What is actually built.....	40
6.2 Target AWS topology.....	40
6.3 Environment and secrets.....	42
6.4 Delivery pipeline.....	42
Technologies Used.....	43
Conclusion.....	48
8.1 Key takeaways.....	48
8.2 Practical applications.....	48
8.3 Limitations.....	49

8.4 Future work.....	49
8.5 Closing.....	50
References.....	51

List of Tables

Table 1.1: Build milestones and their outcomes
Table 2.1: Functional requirements (condensed)
Table 2.2: Non-functional requirements
Table 2.3: Delivered feature set by area
Table 3.1: oRPC procedure surface
Table 4.1: Indexes by collection
Table 5.1: routes.analyze — cold vs. warm cache
Table 5.2: Explore query — 2dsphere index on vs. off
Table 7.1: Core stack and pinned versions

List of Figures

Figure 1.1: Trek Together landing page — the product promise and the four-step flow
Figure 1.2: Component architecture and request flow across the monorepo
Figure 2.1: Use-case diagram — Visitor and Registered User against the ten core use cases
Figure 2.2: Explore — public routes near Shenandoah NP, with distance from the map centre and difficulty badges
Figure 2.3: My Routes — the owner's saved routes with visibility and management actions (FR-5)
Figure 3.1: Class/module diagram — procedures, services, data repositories and integrations
Figure 4.1: Collection/ER diagram — application collections, Better-Auth identity, references and indexes
Figure 5.1: routes.analyze sequence — procedure → service → cache-first elevation → grading, then save + share
Figure 5.2: Route input on the Plan page — five waypoints plotted on the map, ready for routes.analyze
Figure 5.3: The flagship feature in the UI — snapped route, difficulty badge, metrics and elevation profile
Figure 5.4: Shared route view at /r/:id — the persisted analysis rendered from the share link
Figure 5.5: Community trek logs — actual duration and rating beside the algorithmic estimates
Figure 6.1: Target AWS deployment — CloudFront + S3 edge, ALB + ECS Fargate API inside a VPC, MongoDB Atlas

Figure 7.1: Authentication UI (Better-Auth email/password sessions) — the split-screen sign-in screen

Applied Software Project

Abstract

Trek Together is a full-stack, type-safe web application that turns a hand-drawn line on a map into a trustworthy answer to a hiker's most practical question: *how hard is this trail, really?* A user plots a route as a polyline on an interactive map (optionally snapped onto real walking paths); the backend samples real elevation data along the route, computes total ascent and descent, estimates walking time using both Naismith's rule and Tobler's hiking function, and assigns a difficulty grade using the Shenandoah National Park rating system. Routes can be saved, shared publicly by link, exported as offline GPX/JSON itineraries, discovered geospatially ("routes near me"), and enriched with community trek logs that record how long the trail *actually* took.

The system is a MERN-family monorepo: a React 19 + TanStack Router progressive web app, an Express 5 server exposing end-to-end typed oRPC procedures validated with Zod, MongoDB accessed through Prisma, and Better-Auth session authentication. The technically significant backend work is threefold. First, a **geospatial document schema**: route geometry is stored as GeoJSON with a MongoDB 2dsphere index, powering a \$geoNear discovery query. Second, a **cache-first elevation integration**: third-party elevation APIs are slow and quota-limited (~1 request/second, ~1,000 calls/day), so every sampled point is cached in a quantised, TTL-indexed MongoDB collection. Third, a **measured optimisation story**: the cache converts a cold, rate-limit-bound analysis (~576 ms for a 12-vertex ridge route) into a ~2.4 ms warm response — a ~241× improvement with a ~100% hit ratio — while the 2dsphere index reduces the explore query's documents examined from 2,000 (full scan) to ~204.

Beyond the algorithms, the project demonstrates production disciplines: 194 automated tests, typed error degradation, provider fallback with a daily quota circuit breaker, WCAG 2.2 AA accessibility verification, offline-capable PWA caching, Docker deployment artifacts, and an operations runbook. This report documents the requirements, architecture, low-level design, database schema, the flagship feature's development and benchmarks,

the deployment topology, and the technologies used — with every claim backed by artifacts produced during the build.

Project Description

1.1 What the project is

Trek Together is a trek route planner, difficulty grader, and community trek-logging application. The core loop is deliberately short: **plot** → **analyze** → **save** → **share**. A user traces a trail on a Leaflet/OpenStreetMap map by clicking waypoints; the application snaps those waypoints onto real walking paths where possible, samples real elevation along the resulting line, and returns an elevation profile, total ascent and descent in metres, two walking-time estimates (Naismith and Tobler), and a difficulty score with a human-readable band such as *Moderate* or *Strenuous*. A saved route becomes a shareable page that anyone can open without an account, an exportable GPX/JSON itinerary usable offline on the trail, and a discoverable entry in a map-based “routes near me” search. Authenticated users can additionally log completed treks against any visible route — the date, the actual duration, a 1–5 rating, and notes — and every route page aggregates those logs into community statistics that sit alongside the algorithmic estimates.

The name is borrowed from a real trekking community the author hikes with. The app is a lightweight digital companion to that experience: before a group commits a weekend to a trail, someone has to answer how far it is, how much climbing it involves, how long it will take, and whether the slowest member will cope. Today that means juggling a mapping tool, a separate elevation service, a guess, and a spreadsheet. Trek Together collapses the workflow into one page with transparent, citable math.

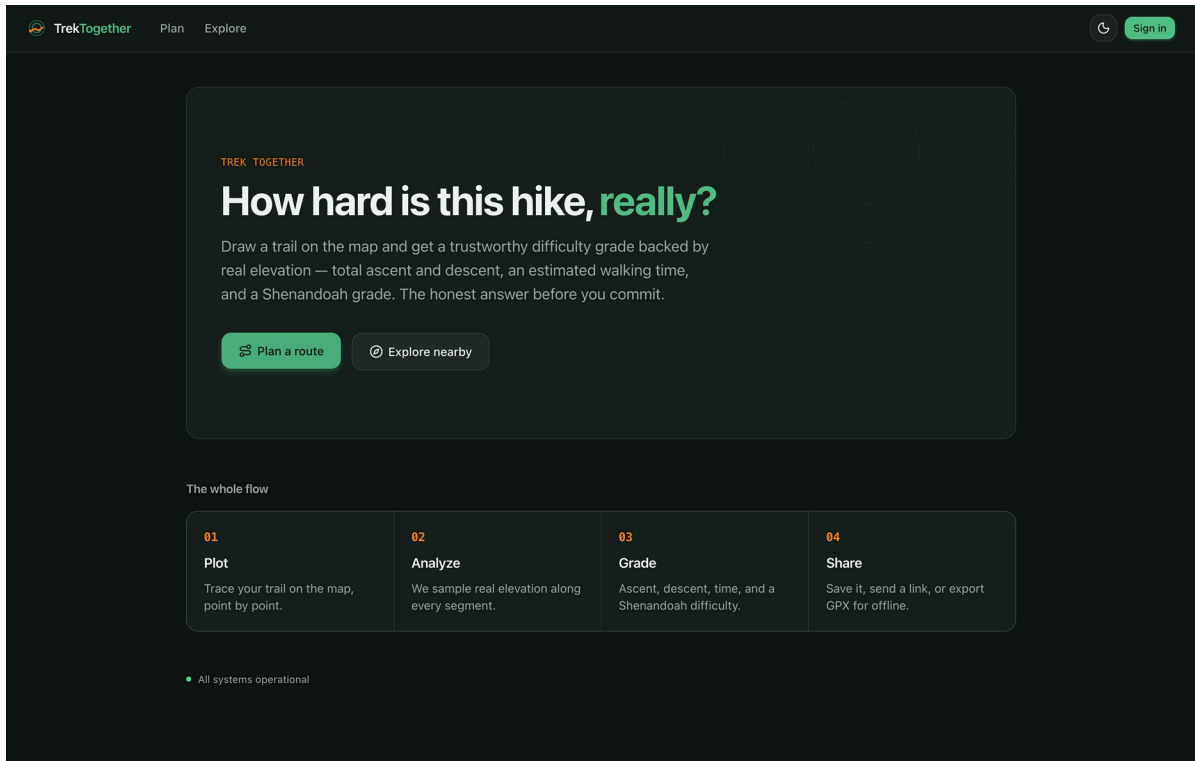


Figure 1.1: Trek Together landing page — the product promise and the four-step flow

1.2 Objectives

The project is the capstone for the Scaler Neovarsity / Woolf MSc in Computer Science (Backend Specialization), and its objectives were fixed in a product requirements document before the first line of code:

- **G1 — Trustworthy analysis.** Turn a hand-drawn route into a reproducible analysis — elevation profile, ascent/descent, time estimate, difficulty grade — using documented, citable formulas rather than black-box numbers.
- **G2 — Fast and quota-safe.** Keep analysis fast despite rate-limited third-party elevation APIs, via a measurable elevation cache. This is the report's headline optimisation.
- **G3 — Save and share.** Persist routes and make public ones viewable by a stable link, with no account required to view.
- **G4 — Geospatial discovery.** Let routes be found by location with a real geospatial query (\$geoNear) backed by a 2dsphere index.
- **G5 — Offline itineraries.** Export self-contained GPX/JSON itineraries and make the app an installable, offline-capable PWA.
- **G6 — Clean backend architecture.** Demonstrate a layered, end-to-end type-safe backend suitable for academic grading.

- **G7 — Snap to real trails.** Snap plotted waypoints onto actual walking paths, falling back to straight lines when routing is unavailable.
- **G8 — Community layer.** Per-route trek logs and aggregate statistics (count, average rating, typical actual vs. predicted time).

1.3 Relevance and real-life application

Every consumer mapping product that touches the outdoors — AllTrails, Komoot, Strava, Gaia GPS — must solve the same three backend problems this project isolates: geospatial storage and querying of route geometry, integration with expensive external geo-data providers behind aggressive caching, and converting raw elevation samples into human-meaningful metrics. The project is therefore a scaled-down but architecturally honest replica of a real product category. The same patterns generalise beyond hiking: food-delivery services run “couriers near me” queries structurally identical to `routes.explore`; weather and geocoding integrations in logistics platforms need the same cache-first, rate-limited client discipline; and any analytics feature that persists derived results (here, the embedded elevation profile) faces the same embed-versus-reference schema decisions documented in the database schema section.

1.4 Development process

The capstone is explicitly graded on an iterative, dated commit history, so the build was organised as a sequence of small milestones, each landing as reviewable Conventional-Commit units (`feat:`, `fix:`, `docs:`, `test:`, `chore:`). Ninety-two commits were made between 27 June 2026 and 12 July 2026 (planning and product documents were authored earlier in the project window), tracked against a 92-task backlog with per-task acceptance criteria. The milestone plan ordered risk deliberately: the elevation cache and the pure analysis math (M1–M2) were built and unit-tested before any HTTP surface existed; the flagship procedure (M3) then composed them; persistence, sharing, discovery, frontend, PWA, documentation, hardening, community logs, and deployment followed.

Table 1.1: Build milestones and their outcomes

Milestone	Goal	Outcome
M0 Scaffold	Monorepo, MongoDB replica set, Prisma, Better-Auth, quality gates	7/8 done (public GitHub push is owner's step)

Milestone	Goal	Outcome
M1 Elevation + cache	Provider clients, cache-first wrapper, batching, rate limiting	Done (7/7), unit-tested
M2 Analysis math	Densify, smooth, ascent/descent, Naismith, Tobler, grading	Done (8/8), pure + unit-tested
M3 routes.analyze	Flagship procedure end-to-end + cold/warm benchmark	Done (6/6); ~241× warm speed-up
M4 Persistence	Route schema, 2dsphere setup, repository, authorization	Done (8/8)
M5 Share + export	Public share-by-link, GPX/JSON export	Done (5/5)
M6 Explore	\$geoNear query + index on/off benchmark	Done (4/4); 204 vs 2,000 docs examined
M7 Frontend	Map planner, profile chart, auth UI, route pages	Done (8/8), browser-verified
M8 PWA & offline	Installable app, shell + tile + data caching	Done (4/4), offline-verified
M9 Docs	README, diagrams, OpenAPI export, benchmarks	Done (7/8); this report is the remaining task
M10 Stabilize & harden	Quota breaker, typed degradation, error/offline UX, a11y	Done (15/15); WCAG 2.2 AA, axe 0 violations
M11 Community logs	TrekLog model, logs.* procedures, stats, UI	Done (7/7)
M12 Deploy-ready	Dockerfile, production compose, runbook	Done (4/4), Docker-verified
M13 Optimization pass	Query caching, authz projection gate, dependency hygiene	Done (1/1), gates green

Quality gates ran on every commit: Biome (lint + format) via a Husky pre-commit hook, TypeScript strict type-checking, and a Vitest suite that grew to **194 tests**. Continuous integration (GitHub Actions) repeats lint, type-check, test, and build on every push.

1.5 System overview

The system is a pnpm + Turborepo monorepo with two applications and six shared packages. The web app (apps/web) is a React 19 + TanStack Router single-page PWA;

the server (apps/server) is a thin Express 5 entrypoint that mounts three surfaces: the typed oRPC handler at /rpc, Better-Auth at /api/auth, and a generated OpenAPI reference at /api-reference. All API logic lives in the packages/api package, organised into four strict layers — thin procedures, pure services, a data layer that owns every database access, and integrations that wrap external providers behind a cache. The class-diagram section details the layering; Figure 1.2 shows the component topology and request flow.

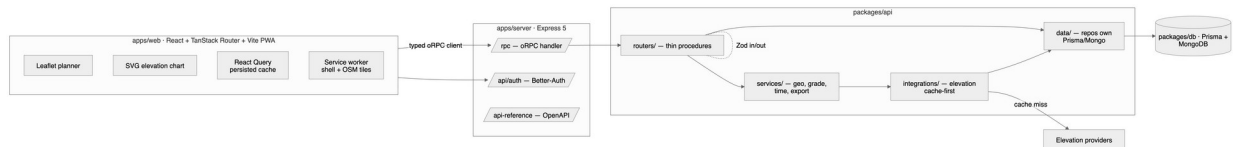


Figure 1.2: Component architecture and request flow across the monorepo

Two properties of this architecture carry most of the engineering weight. First, **end-to-end type safety**: the web client's types are inferred from the server's router definition, so a change to a procedure's Zod schema propagates to the frontend at compile time — there is no hand-maintained API contract to drift. Second, **choke-point integrations**: every elevation lookup in the entire system flows through one cache-first function, which is what makes the quota math of the feature-development section enforceable rather than aspirational.

Requirement Gathering

2.1 Users and personas

The product deliberately has just two actors. A **User** is an authenticated account holder who plots routes (snapped to real trails), runs analyses, saves and manages routes, toggles their visibility, browses public routes near a location, exports itineraries, and logs completed treks on any route they can see. A **Viewer** is an anonymous person with a link: they can open a shared route, read its map, elevation profile, difficulty, time estimates and community logs, and export its itinerary — with no account. An admin/moderator role is explicitly out of scope; there is no content-moderation surface in v1.

2.2 Use cases

The use-case diagram in Figure 2.1 maps both actors onto the ten core use cases. Note the two dependency edges: saving a route requires an analysis (a route is persisted *with* its derived metrics), and analysis itself depends on the external elevation API through the cache.



Figure 2.1: Use-case diagram — Visitor and Registered User against the ten core use cases

Representative user stories, written before implementation and used to derive the functional requirements:

- As a User, I can plot a route by clicking points on a map and see the line update live, snapped onto real walking paths when possible.
- As a User, I can analyze a plotted route and see its elevation profile, total ascent/descent, estimated time (Naismith and Tobler), and difficulty grade.
- As a User, I can save a route with a name and description, mark it public or private, and copy a share link anyone can open.
- As a User, I can browse public routes near a chosen location, ordered by distance.
- As a User, I can log a completed trek — date, actual duration, a 1–5 rating, notes — on any route I can see.
- As a Viewer, I can open a shared link and view the route, its profile, difficulty, time, and community logs without signing in, and export the itinerary as GPX/JSON.

2.3 Functional requirements

Each functional requirement was specified with acceptance criteria; Table 2.1 summarises all ten. The wording below is condensed from the product requirements document that governed the build.

Table 2.1: Functional requirements (condensed)

ID	Requirement	Key acceptance criteria
FR-1	Authentication	Email/password with sessions (Better-Auth). Write and owned-data procedures reject unauthenticated requests; sessions persist across reloads.
FR-2	Route plotting	Clicking adds vertices; line + running distance update live; geometry captured as GeoJSON LineString.
FR-3	Route analysis (flagship)	Polyline densified to ~30–90 m spacing; cache-first elevation lookup; response includes profile, ascentM, descentM, distanceM, both time estimates, difficulty score + band; Zod-validated.
FR-4	Elevation caching	Quantised-key cache collection; unique + TTL indexes; batched (≤ 100 pts/request) and rate-limited (≤ 1 req/s) provider calls; repeat analysis serves fully from cache; cold vs. warm measurable.
FR-5	Save & manage routes	Saved route persists all derived fields; only the owner can edit/delete; paginated listing.
FR-6	Share by link	routes.getById returns a public route to an anonymous caller; private routes never exposed; uniform NOT_FOUND hides existence.

ID	Requirement	Key acceptance criteria
FR-7	Explore nearby	Given location + radius, returns public routes by proximity; plan uses the 2dsphere index (verified via explain); paginated, capped limit.
FR-8	Offline export + PWA	Valid GPX and JSON containing geometry, profile and metrics; exported file needs no network; PWA installs and serves a viewed itinerary offline.
FR-9	Trail snapping	2–25 waypoints snapped onto walking paths; missing token or no path yields typed ROUTING_UNAVAILABLE and a straight-line fallback so planning always works.
FR-10	Community trek logs	logs.create (auth) validates date not in future, $0 < \text{duration} \leq 7$ days, integer rating 1–5, notes $\leq 2,000$ chars; visibility-gated; logs.listForRoute returns paginated logs + {count, avgRating, avgActualDurationS}, anonymously for public routes.

2.4 Non-functional requirements

Table 2.2: Non-functional requirements

ID	Category	Requirement
NFR-P1	Performance	Warm (fully cached) analysis returns quickly — provisional target p95 < 300 ms, replaced by measurement (achieved: p95 ≈ 2.9 ms).
NFR-P2	Performance	Cold analysis latency is bounded by the provider's rate limit; the cold figure is recorded, not hidden.
NFR-P3	Performance	Headline optimisation: cache hit ratio and cold-vs-warm latency are measured and reported.
NFR-P4	Performance	The explore query is benchmarked with vs. without the 2dsphere index via explain(executionStats).
NFR-S1	Security	Every oRPC input is Zod-validated; raw Mongo geo commands never concatenate user input.
NFR-S2	Security	Writes and owned-data reads require a valid session; only isPublic routes are exposed anonymously; missing and forbidden are indistinguishable (NOT_FOUND).
NFR-S3	Security	No secrets in the repository; elevation calls always pass the cache wrapper and respect provider quotas; OSM tile policy honoured.

ID	Category	Requirement
NFR-R1	Reliability	Elevation failures fall back to the secondary provider and/or served cache; unresolvable failures return a typed ELEVATION_UNAVAILABLE.
NFR-R2	Reliability	MongoDB runs as a replica set (required by Prisma transactions), single-node in development.
NFR-U1	Usability	Installable PWA; service worker caches the app shell and viewed itineraries for offline use.
NFR-U2	Accessibility	WCAG 2.2 AA — verified with axe-core across key routes in light and dark themes (0 violations after fixes).
NFR-M1	Maintainability	Strict TypeScript, layered architecture, unit-tested domain math with known fixtures.

2.5 Feature set

Table 2.3: Delivered feature set by area

Area	Delivered capability
Plan	Leaflet map planner: click-to-plot waypoints, snap-to-trail via routing provider, undo/clear, place search (geocoding), “use my location”.
Analyze	Elevation profile chart, ascent/descent, Naismith + Tobler time, Shenandoah grade badge with numeric score.
Save & share	Authenticated CRUD on routes; public/private toggle; stable share URL /r:id viewable anonymously.
Export	GPX 1.1 with elevations and a versioned JSON itinerary (documented schema); both fully offline-usable.
Explore	“Routes near me” map + list via \$geoNear, distance-sorted, with place search and geolocation.
Community	Per-route trek logs (date, actual duration, 1–5 rating, notes) with aggregate stats: count, average rating, average actual duration vs. prediction.
Offline / PWA	Installable app; precached shell; OSM tiles CacheFirst (bounded); persisted query cache lets a viewed route render fully offline.
Accessibility	WCAG 2.2 AA verified (axe-core, 0 violations); keyboard-reachable controls; documented pointer-only map limitation.
Operations	Multi-stage Dockerfile, production docker-compose with replica-set Mongo, health/readiness endpoint, runbook, CI pipeline.

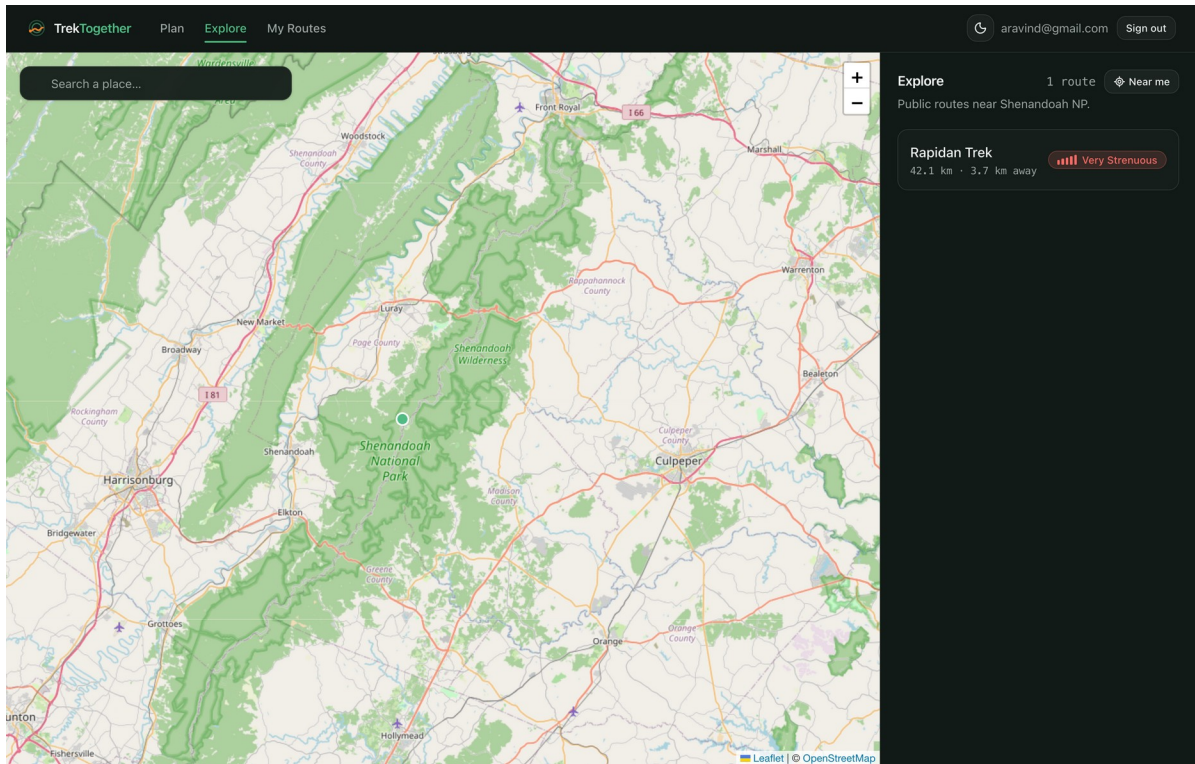


Figure 2.2: Explore — public routes near Shenandoah NP, with distance from the map centre and difficulty badges

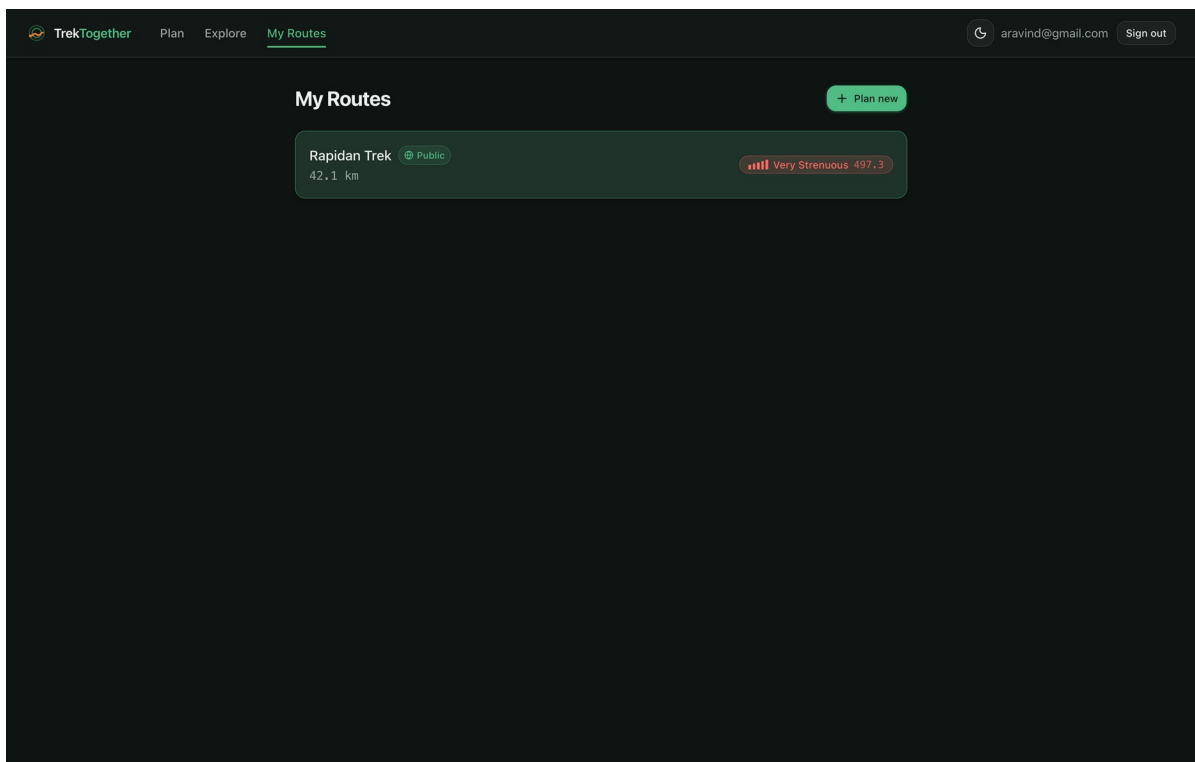


Figure 2.3: My Routes — the owner's saved routes with visibility and management actions (FR-5)

Class Diagrams

3.1 The layering

The backend follows a strict four-layer design. Each layer has one responsibility and one direction of dependency; the rules were enforced throughout the build and are visible in the class diagram of Figure 3.1.

- **Procedures** (`packages/api/src/routers/`) are thin oRPC adapters: validate input with Zod, call a service or repository, map domain errors to typed API errors, return typed output. No math, no direct database access.
- **Services** (`services/`) hold the domain logic — geodesy, profile building, smoothing, ascent/descent, Naismith/Tobler time, Shenandoah grading, GPX/JSON export. They are pure, framework-free, database-agnostic TypeScript functions, unit-tested against known fixtures.
- **Data layer** (`data/`) owns *every* database access: the routes repository, the trek-log repository, and the elevation-cache repository. Geospatial queries Prisma cannot express run here through raw MongoDB commands (`aggregateRaw`), never from services.
- **Integrations** (`integrations/`) wrap external HTTP providers — two elevation APIs, a routing (snap-to-trail) provider, and a geocoding provider — behind small typed clients. The elevation client is cache-first and is the single choke-point for all elevation traffic.

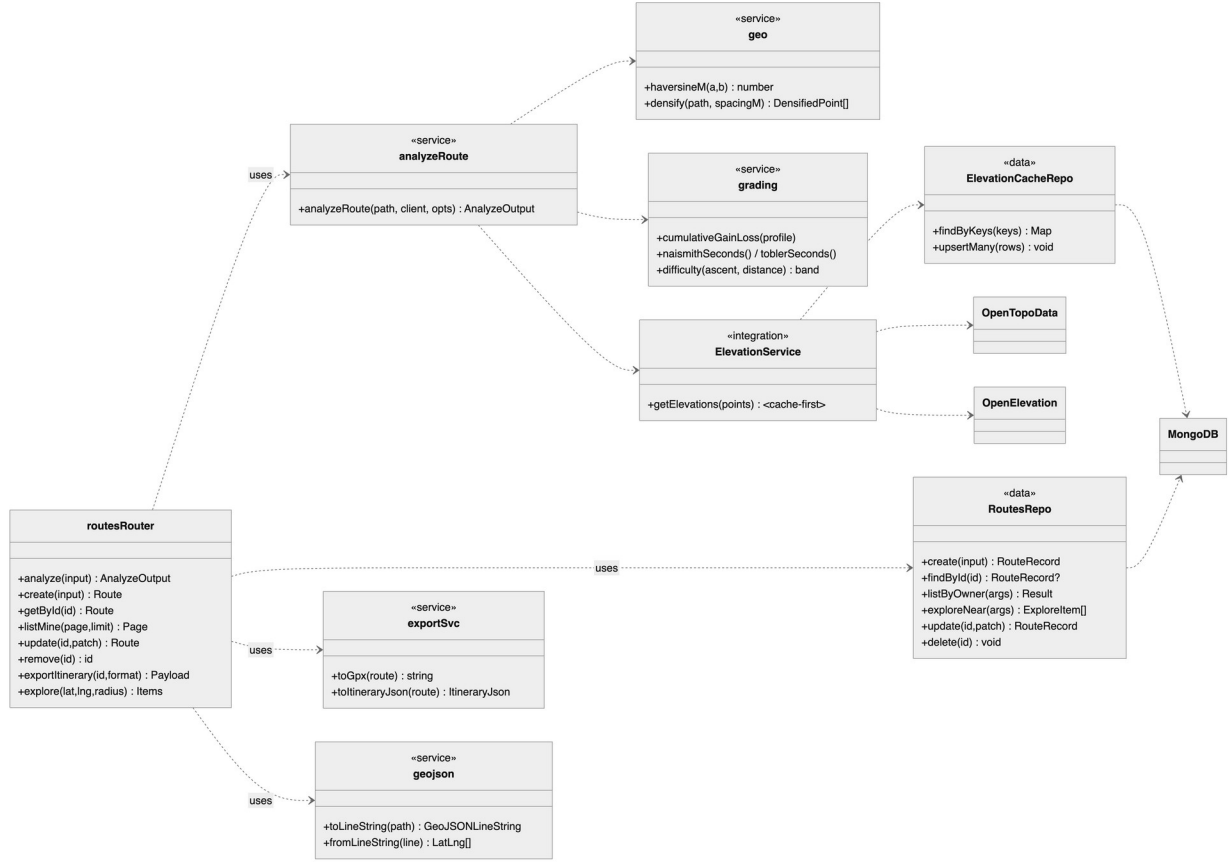


Figure 3.1: Class/module diagram — procedures, services, data repositories and integrations

3.2 Modules in the diagram

routesRouter exposes the nine route procedures (Table 3.1). Its `analyze` handler is representative of the layer's thinness: it builds the cache-backed elevation client from the request context, delegates to the `analyzeRoute` service, and translates the three domain failure modes into typed errors — `RouteTooLargeError` → `VALIDATION`, `ElevationCoverageError` (a route over the ocean or outside dataset bounds — a user-input problem) → `VALIDATION`, and `ElevationUnavailableError` (providers down or quota exhausted — a transient outage) → `ELEVATION_UNAVAILABLE` carrying the count of unresolved points. Anything unexpected propagates to the central Express handler and surfaces as `INTERNAL` without leaking detail.

analyzeRoute is the composition root of the flagship feature: `buildProfile` → `smoothProfile` → `cumulativeGainLoss` → `naismithSeconds` + `toblerSeconds` → `difficulty`, a pipeline whose order is pinned by a dedicated test.

It receives the elevation client by dependency injection, so unit tests exercise the full pipeline with a fake client and no network or database. A length-based guard rejects routes that would densify past 5,000 sample points *before* any lookup happens, protecting the provider quota from a single oversized request.

geo / grading / geojson / exportSvc are the pure services: haversineM great-circle distance, densify polyline resampling, cumulative gain/loss with smoothing, the two time models, the Shenandoah score, GeoJSON conversion (including the [lng, lat] ordering MongoDB requires), and the two export formats.

RoutesRepo / ElevationCacheRepo / LogsRepo encapsulate Prisma models plus raw-Mongo operations: `exploreNear` runs the `$geoNear` aggregation; `ElevationCacheRepo` exposes exactly two operations (`findByKey`, `upsertMany`), which is the entire surface the cache needs; the logs repository computes per-route aggregate statistics with a `$match` → `$group` aggregation.

ElevationService is the cache-first reader described in depth in the feature-development section; it consults `ElevationCacheRepo`, batches misses to the primary provider (OpenTopoData) and falls back to the secondary (Open-Elevation) on provider errors.

3.3 API surface

Table 3.1: oRPC procedure surface (all inputs Zod-validated; all outputs typed)

Procedure	Auth	Purpose
<code>routes.snap</code>	optional	Snap ≤25 waypoints onto real walking paths; typed <code>ROUTING_UNAVAILABLE</code> + straight-line fallback
<code>routes.analyze</code>	optional	Flagship: polyline → profile, ascent/descent, Naismith + Tobler time, difficulty (feature-development section)
<code>routes.create</code>	required	Persist a route with its full analysis result
<code>routes.getById</code>	public if <code>isPublic</code>	Shared/public route view; uniform <code>NOT_FOUND</code> for missing or forbidden
<code>routes.listMine</code>	required	Owner's routes, paginated
<code>routes.update</code>	owner	Edit name/description/visibility
<code>routes.remove</code>	owner	Delete a route

Procedure	Auth	Purpose
routes.explore	optional	Public routes near a point — \$geoNear, distance-sorted, paginated
routes.exportItinerary	route visibility	GPX 1.1 or versioned JSON itinerary
logs.create	required	Log a completed trek on a visible route; author identity from the session, never the client
logs.listForRoute	route visibility	Paginated newest-first logs + aggregate stats
places.search	optional	Geocode a place name for the map search box
healthCheck	public	Liveness probe

3.4 Cross-cutting design decisions

Request context. Every procedure receives { `db`, `session`, `requestId` } — the Prisma client, the Better-Auth session (or null), and a correlation id that flows into structured access logs and provider-switch warnings. `protectedProcedure` is a middleware wrapper that rejects anonymous callers with `UNAUTHORIZED` before the handler runs.

Uniform authorization gate. All visibility decisions go through one shared `findVisibleRoute` helper: a route is visible if it is public or owned by the caller; *missing* and *someone else's private* both surface as the same `NOT_FOUND`, so the existence of a private route never leaks. The trek-log procedures reuse the same gate, keeping the community layer's authorization identical to the routes'.

Typed errors end-to-end. Because `oRPC` contracts include error shapes, the web client can switch on `ELEVATION_UNAVAILABLE` or `ROUTING_UNAVAILABLE` and render precise degraded states (e.g. “analysis briefly unavailable — cached routes still work”) instead of a generic failure toast.

Units in names. Every field carries its unit — `distanceM`, `ascentM`, `actualDurationS`, `estTimeNaismithS`. Internals are SI (metres, seconds); conversion to feet/miles happens only at the grading boundary (a documented grading decision) and

to $\text{km/h}\cdot\text{min}$ at the UI edge. This convention eliminated an entire class of unit bugs during the build.

Database Schema Design

4.1 Document model overview

The database is MongoDB, accessed through Prisma (`provider = "mongodb"`), with a multi-file schema. Four Better-Auth collections (`user`, `session`, `account`, `verification`) are created and managed by the auth library's Prisma adapter; the application adds three collections of its own: `routes`, `elevationCache`, and `trekLogs`. Figure 4.1 shows the collection diagram; the textual schema follows.

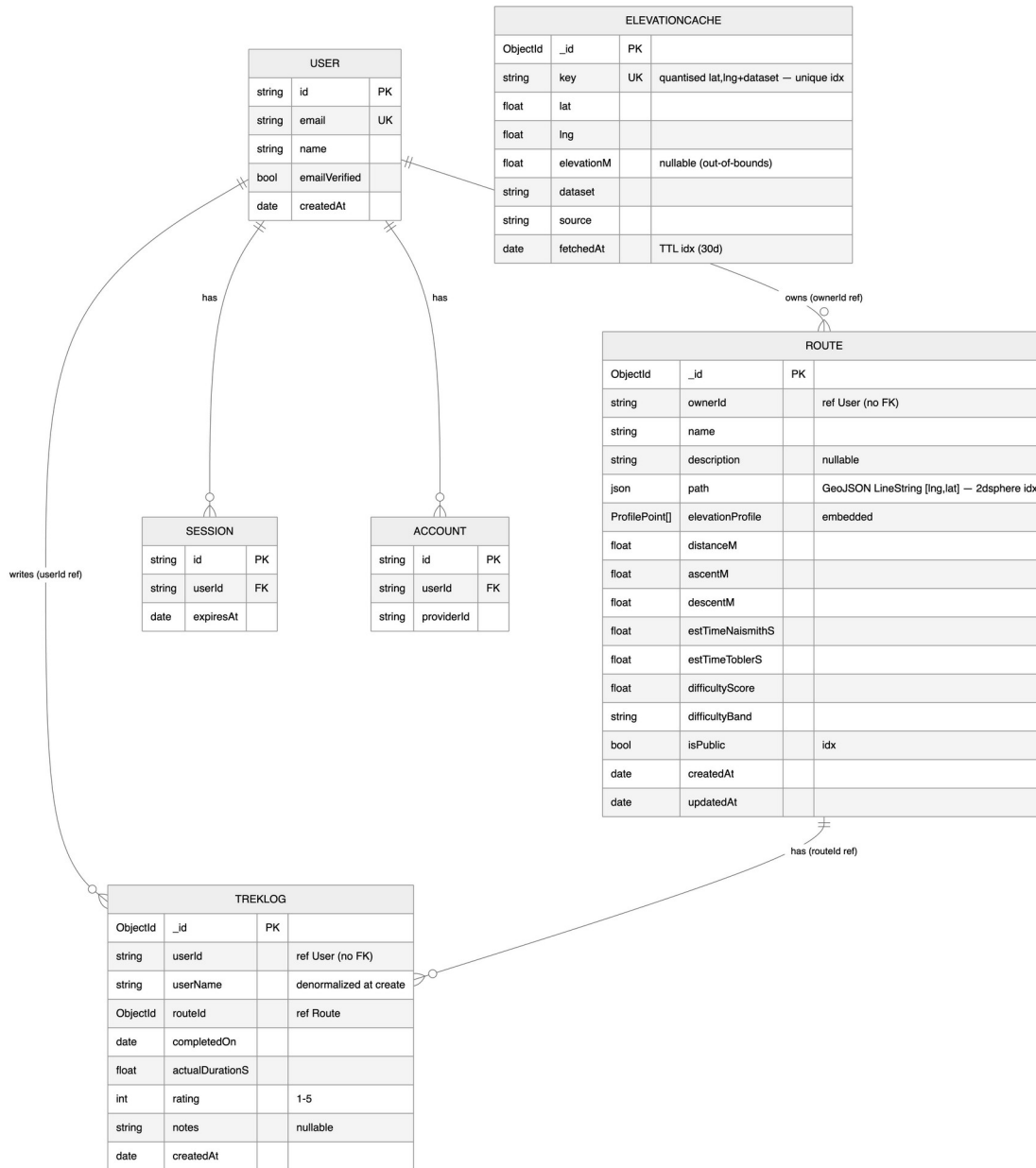


Figure 4.1: Collection/ER diagram — application collections, Better-Auth identity, references and indexes

4.2 Schema (textual)

Collections and fields as implemented in the Prisma schema (types abbreviated; all timestamps are UTC):

```

routes
  _id          ObjectId (PK)
  ownerId      String — Better-Auth user id (referenced, no
FK)
  name         String

```

```

    description      String?
    path             Json — GeoJSON LineString { coordinates:
[[lng,lat], ...] }
    elevationProfile ProfilePoint[] — EMBEDDED [{ distanceAlongM,
elevationM }]
    distanceM, ascentM, descentM      Float
    estTimeNaismithS, estTimeToblers Float
    difficultyScore   Float; difficultyBand String
    isPublic          Boolean (default false)
    createdAt, updatedAt      DateTime

elevationCache
  _id      ObjectId (PK)
  key      String — "<dataset>:<qlat>,<qlng>" (UNIQUE)
  lat, lng Float — grid-centre coordinates
  elevationM Float? — null preserved for out-of-bounds points
  dataset   String — logical namespace baked into key (e.g.
srtm30m)
  source     String — provider that actually answered (provenance)
  fetchedAt  DateTime — drives the 30-day TTL index

trekLogs
  _id      ObjectId (PK)
  userId   String — referenced;  userName String —
denormalized at create
  routeId  ObjectId — referenced
  completedOn DateTime; actualDurationS Float; rating Int (1-
5)
  notes    String?; createdAt DateTime

```

4.3 Embed vs. reference — the design decisions

MongoDB offers a per-field choice between embedding data in a document and referencing it by id; getting those choices right is the central schema-design exercise of a document database. Each Route field fell to a different answer, recorded as an architecture decision during the build:

- **`path` — GeoJSON in a `Json` field.** Prisma has no geometry type, so the LineString is stored as GeoJSON with [lng, lat] coordinate order (the GeoJSON/2dsphere convention). Validity is enforced in application code (Zod plus the geojson service), not by the ORM.
- **`elevationProfile` — embedded.** The profile is read together with the route on every view and export, is bounded in size (the analysis caps sample count), and is never queried independently. Embedding makes a route view a *single document read* — no join, no second round-trip. This is the canonical embed case.
- **`ownerId` — referenced.** A user and their routes have independent lifecycles, and MongoDB enforces no foreign keys regardless; storing the Better-Auth user id as a plain indexed string keeps the reference explicit without coupling the auth schema to

the app schema. Ownership integrity is an application-layer invariant, covered by the authorization tests.

- **`trekLogs` — referenced both ways, with one denormalization.** Logs reference `userId` and `routeId` as plain ids (independent lifecycles, unbounded growth — the anti-embed case), but denormalize `userName` at creation time so listing a route's logs never needs a `$lookup` into the auth collection. The accepted trade-off: a renamed user's old logs keep the old display name.

Persisting the full analysis result on the route is itself a schema decision: a route view or GPX export never recomputes anything and never touches the elevation providers — reads are one document fetch, and the elevation quota budget is spent only when geometry actually changes.

4.4 Indexes — including the two Prisma cannot express

Table 4.1: Indexes by collection

Collection	Index	Type	Purpose
routes	path	2dsphere (out-of-band)	\$geoNear explore query — mandatory for the geo stage
routes	ownerId	B-tree	Owner-scoped listMine
routes	isPublic	B-tree	Public visibility filters; the no-index fallback plan in the benchmark
routes	difficultyScore	B-tree	Difficulty filtering/sorting
elevationCache	key	unique	Cache-key lookups and upsert dedup
elevationCache	fetchedException	TTL 30 days (out-of-band)	Auto-expiry of stale elevation samples
trekLogs	(routeId, createdAt desc)	compound	Newest-first log listing per route
trekLogs	userId	B-tree	Per-user log access

Two of these indexes cannot be declared in the Prisma schema at all, which is one of the two Prisma-on-MongoDB gotchas this project documents as real engineering constraints:

Gotcha 1 — no migrations. Prisma Migrate is unsupported on MongoDB. The schema is synchronised with `prisma db push`; there is no SQL-style migration history. The project

therefore treats the Prisma schema files as the single source of truth and wires `db push` into a scripted flow.

Gotcha 2 — geospatial is not first-class. Prisma's schema language has no geometry type, no 2dsphere index, and no TTL index. Both are created out-of-band by an idempotent setup script (`packages/db/src/setup-indexes.ts`) that runs `createIndex` through the raw command interface. Crucially, `db push` reconciles indexes to the schema and *silently drops* indexes it does not manage — so the setup script is chained onto the project's `db : push` command, and the server verifies both indexes at boot, logging a loud `MISSING INDEX` warning if either is absent. Without the 2dsphere index `routes.explore` fails outright (`$geoNear` requires it); without the TTL index the elevation cache silently stops expiring.

Replica set requirement. Prisma's MongoDB connector requires a replica set (it uses transactions internally, as does Better-Auth). Development runs a single-node replica set in Docker with a self-initiating healthcheck; production mirrors the same topology (see the deployment section).

Feature Development Process

This section deep-dives one key feature end-to-end, as the template requires: ``routes.analyze``, the flagship procedure that converts a plotted polyline into an elevation profile, ascent/descent totals, two time estimates, and a difficulty grade — and the **elevation cache** that makes it fast and quota-safe. It covers the request payload, the layered flow that picks it up, the domain math, the cache's design, and the measured performance improvement: a **$\sim 241\times$ faster** warm response, plus a secondary indexing benchmark for the geospatial explore query.

5.1 The request flow

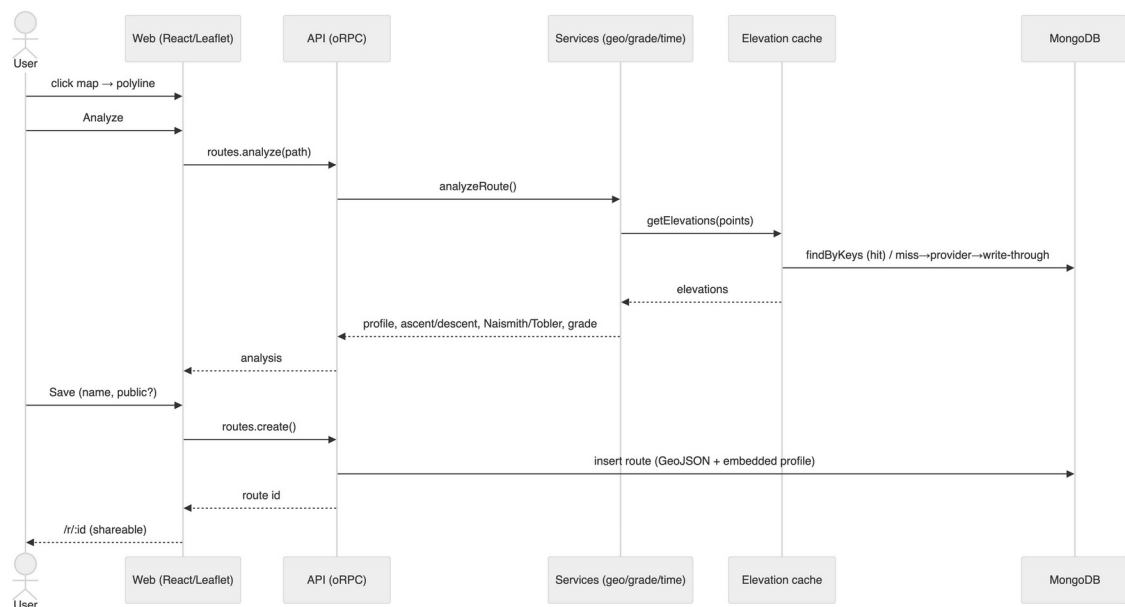


Figure 5.1: routes.analyze sequence — procedure → service → cache-first elevation → grading, then save + share

API request payload. The client calls `POST /rpc/routes/analyze` with an ordered array of vertices and an optional sample spacing. The Zod schema at the boundary constrains every field: latitude $\in [-90, 90]$, longitude $\in [-180, 180]$, 2–3,000 vertices, spacing 10–1,000 m (default 60 m). A representative payload:

```
{
  "path": [
    { "lat": 38.5450, "lng": -78.3230 },
    { "lat": 38.5312, "lng": -78.2957 },
    { "lat": 38.5178, "lng": -78.2704 }
  ]
}
```

```

    ],
    "spacingM": 60
  }

```

Service that picks it up. The procedure is deliberately thin (validation, dependency wiring, error mapping); the `analyzeRoute` service owns the pipeline. Its stages, in pinned order:

- **Guard.** A cheap haversine length estimate rejects routes that would densify past 5,000 sample points before any network or database work — one oversized request cannot drain the day's provider quota.
- **Densify.** The polyline is resampled to one point every ~60 m (configurable 10–1,000 m), matching the ~30–90 m resolution of SRTM elevation data. Sampling sparser than the dataset misses climbs; denser wastes quota without adding information.
- **Elevation lookup (cache-first).** All sampled points go to the elevation service in one call — the single choke-point described in §5.2.
- **Smooth.** Raw SRTM data carries $\pm$ metre-level noise that, summed naively, inflates ascent by 15–25% on rolling terrain. A 5-point centred moving average followed by a 3 m minimum-change threshold flattens the noise; on a synthetic flat section with ± 4 m noise the pipeline reduces reported ascent from 32 m to the correct 0 m, and on a real 2 km climb it corrects ~245 m raw to ~215 m.
- **Ascent/descent.** Cumulative gain and loss are the sums of positive and negative deltas over the smoothed profile.
- **Time.** Naismith's rule as the baseline — 1 hour per 5 km plus 1 hour per 600 m of ascent — and Tobler's hiking function as the slope-aware refinement, $W = 6 \cdot e^{(-3.5 \cdot |S + 0.05|)}$ km/h integrated per profile segment, where S is rise over run. Both estimates are returned and stored.
- **Grade.** The Shenandoah National Park rating: $\text{difficulty} = \sqrt{(2 \times \text{elevation gain} \times \text{distance})}$, computed on the NPS unit basis (gain in feet, distance in miles — the SI internals convert only at this boundary, a documented decision), then banded: <50 Easiest, 50–100 Moderate, 100–150 Moderately Strenuous, 150–200 Strenuous, >200 Very Strenuous.

The response carries the smoothed profile (`[{ distanceAlongM, elevationM }]`), `distanceM`, `ascentM`, `descentM`, `estTimeNaismithS`, `estTimeToblerS`, `difficultyScore`, `difficultyBand`, and a meta block with the request's cache hits and misses — the same counters the benchmark reads. Figure 5.3 shows the feature in the product.

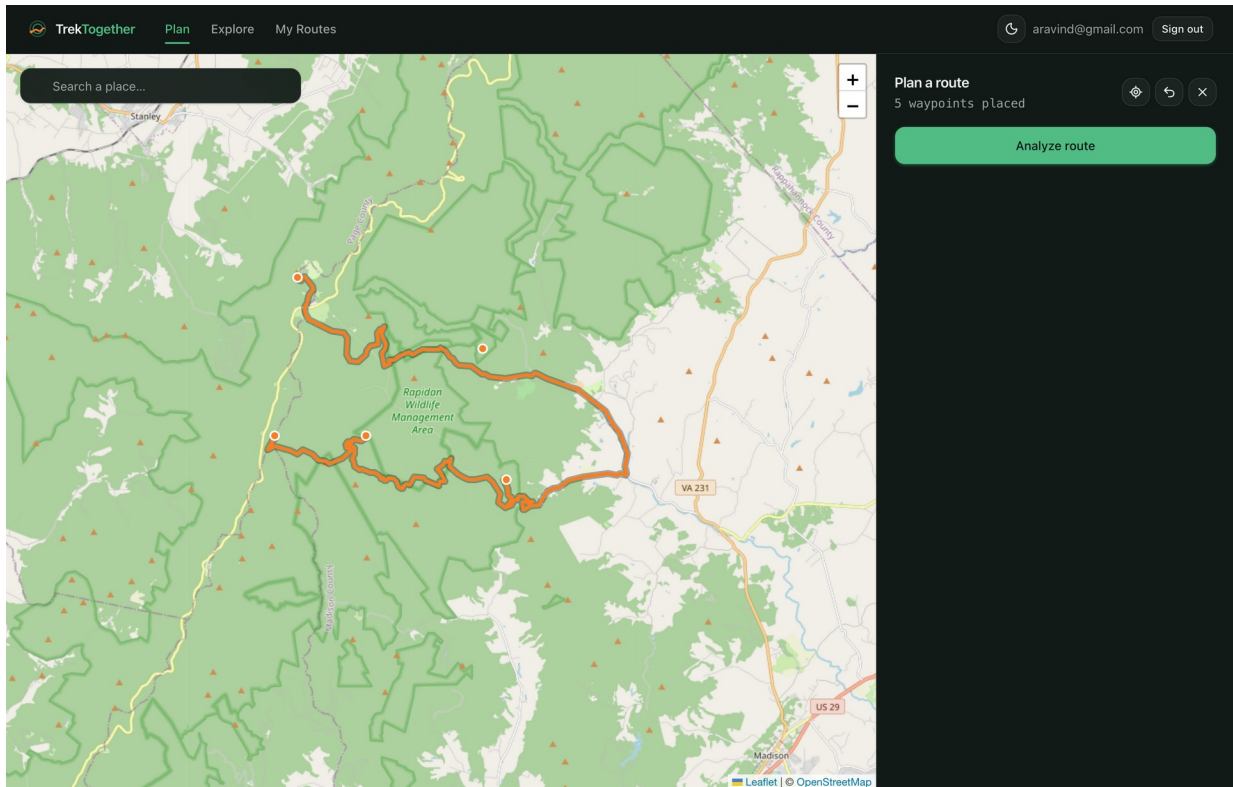


Figure 5.2: Route input on the Plan page — five waypoints plotted on the map, ready for routes.analyze

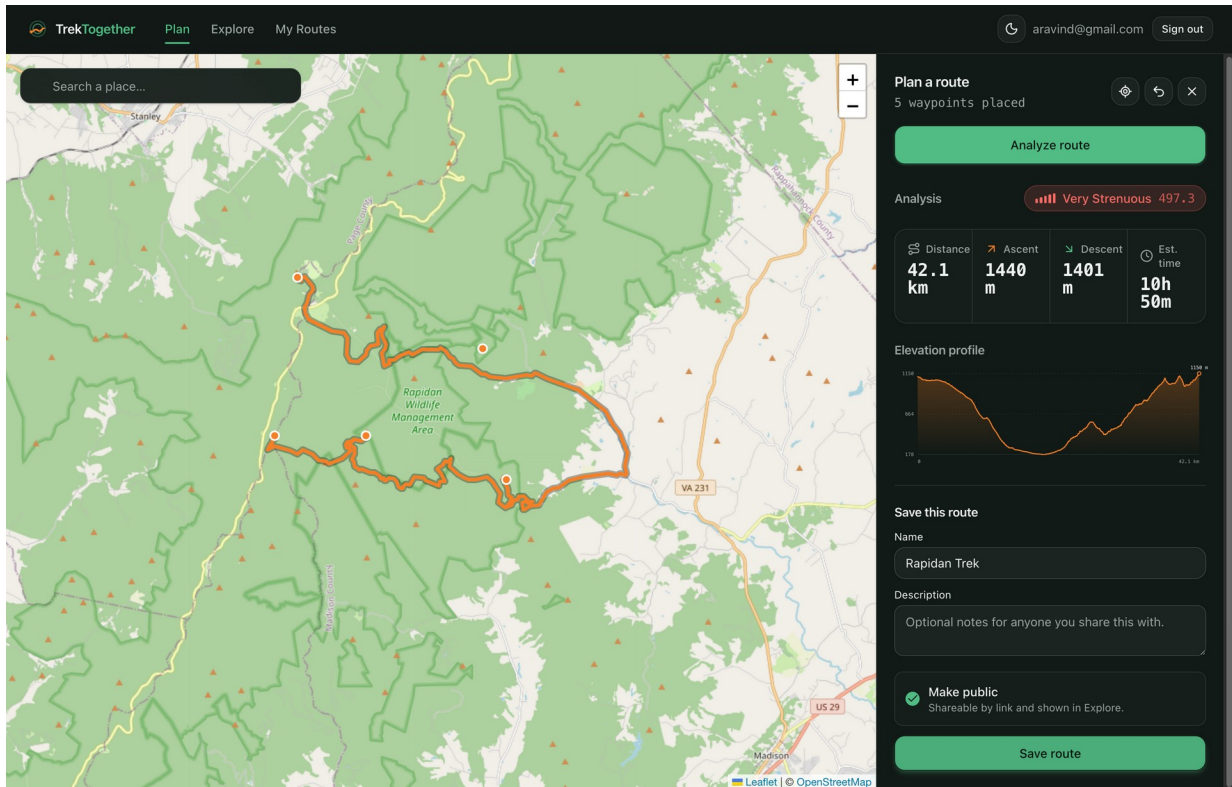


Figure 5.3: The flagship feature in the UI — snapped route, difficulty badge, metrics and elevation profile

5.2 The elevation cache — design

Elevation is the only expensive input, and its economics dictate the architecture. The public OpenTopoData host allows roughly 1 request/second, 100 locations per request, and ~1,000 calls/day per IP; the fallback Open-Elevation public host tolerates about 1,000 requests per month. Uncached, a single 10 km route analysis (~170 sample points, 2 batches) costs seconds of rate-limited latency, and fewer than a thousand analyses would exhaust a whole day's quota. The cache turns those numbers around:

- **Quantisation.** Every coordinate is snapped to a ~1-arc-second grid and keyed as "<dataset>:<qlat>,<qlng>". Nearby requests — the same trail plotted by different users, or the same user re-analyzing after an edit — collapse onto the same keys and hit cache.
- **Read → fetch misses → write-through.** The service deduplicates keys within the request, reads the cache in one `findByKeys`, fetches only the missing points from the provider (batched $\leq 100/\text{request}$, globally rate-limited $\leq 1 \text{ req/s}$), writes them back with `upsertMany`, and merges results into input order.

- **Presence, not truthiness.** A cached null elevation (a point outside the dataset, e.g. over water) is a legitimate hit and is never re-fetched — without this, every analysis touching a lake would re-spend quota forever.
- **Unique + TTL indexes.** The unique key index makes upserts idempotent under concurrency; the 30-day TTL on `fetchAt` bounds staleness and storage without any batch cleanup job.
- **Provider fallback + typed degradation.** A provider error triggers one fallback (OpenTopoData → Open-Elevation); if all providers fail, the service returns what the cache *did* resolve inside a typed `ElevationUnavailableError`, and the client renders a precise degraded state rather than a blank error.
- **Quota circuit breaker.** A process-global limiter serialises all outbound batches (the whole server, not each request, honours 1 req/s); batches run sequentially and fail fast; 429/5xx responses get one polite retry honouring `Retry-After`; and a local per-day counter (default 1,000, configurable) fails fast *without touching the network* once the provider's daily ceiling is reached. At the ceiling, cold analyses degrade to the typed error until the UTC day rolls over — warm analyses are unaffected.

5.3 Headline benchmark — cold vs. warm

The benchmark script analyzes a representative Shenandoah ridge route (12 vertices, densified to 42 unique cache keys at 60 m spacing) twice: **cold**, after clearing the cache collection — every point is a rate-limited provider fetch — then **warm**, 10 iterations against the populated cache with zero provider calls. Cold is reported as a single figure (it is bounded by the upstream API, not by local computation); warm is summarised as p50/p95. Run of 10 July 2026, provider OpenTopoData, Node v24.17.0:

Table 5.1: routes.analyze — cold vs. warm cache (bench-analyze.ts, 2026-07-10)

Scenario	Latency	Cache behaviour
Cold (empty cache)	575.6 ms	42 misses / 0 hits — every point fetched, rate-limited
Warm p50	2.4 ms	100% hit ratio — zero provider calls
Warm p95	2.9 ms	—

Warm is ~241× faster than cold. The two numbers also scale differently: cold latency grows with route length × the provider's rate limit (a 5,000-point route costs 50 rate-limited batches ≈ 50+ seconds), while warm latency grows only with local computation and one indexed cache read — the p95 of 2.9 ms comfortably beats the provisional 300 ms target by two orders of magnitude. Equally important is what the cache does to *capacity*: at

~1,000 provider calls/day the uncached system supports at most a few hundred analyses per day in total, whereas the cached system spends quota only on never-before-seen terrain and serves repeat traffic — the common case for a shared trail catalogue — for free. The cache is therefore not a micro-optimisation; it is what makes the product viable on free-tier data sources.

5.4 Secondary benchmark — the 2dsphere index

The explore feature (“public routes near a point”) is a `$geoNear` aggregation over route geometry. To quantify the index's effect, 2,000 public routes were seeded across a ~1.8° box around Shenandoah and the same 25 km query was measured with and without the geospatial index. Because `$geoNear` refuses to run at all without a 2dsphere index, the on/off contrast uses the equivalent `$geoWithin/$centerSphere` filter through `explain("executionStats")`; the production `$geoNear` plan was then verified with the index present (winning stage `GEO_NEAR_2DSPHERE`).

Table 5.2: Explore query — 2dsphere index on vs. off (bench-explore.ts, 2026-07-10, 2,000 seeded routes)

Scenario	Index used	Documents examined	p50 latency
2dsphere ON	path_2dsphere	204	2.09 ms
2dsphere OFF	isPublic B-tree fallback	2,000 (full scan)	3.54 ms

Both plans return the same 139 matching routes, but the geo index examines ~10% of the documents — an index seek versus an $O(n)$ post-filter. At 2,000 documents the wall-clock gap is modest (~1.7×); the durable signal is the **documents-examined** column, which grows linearly with collection size for the scan and stays proportional to the result set for the seek. The benchmark preserves both the method and the explain output so the claim is reproducible (`pnpm -F server bench:explore`).

5.5 Verification

The feature is covered at three levels. Unit tests fix the math against known fixtures (a route with a known ascent must produce a known Naismith time and a known grade; the

smoothing stages have before/after fixtures; the cache service is tested with a fake provider for hit/miss/fallback/degradation paths, including the cached-null rule). Integration tests exercise the procedures over HTTP with a live database — validation failures, authorization (a private route's analysis fields never leak), pagination caps, and the typed error contracts. The two benchmark scripts are kept out of the default test run (they need a live database and, cold, a real provider) but are one command each to re-run, and they rewrite their own dated results files so the report's numbers stay honest.

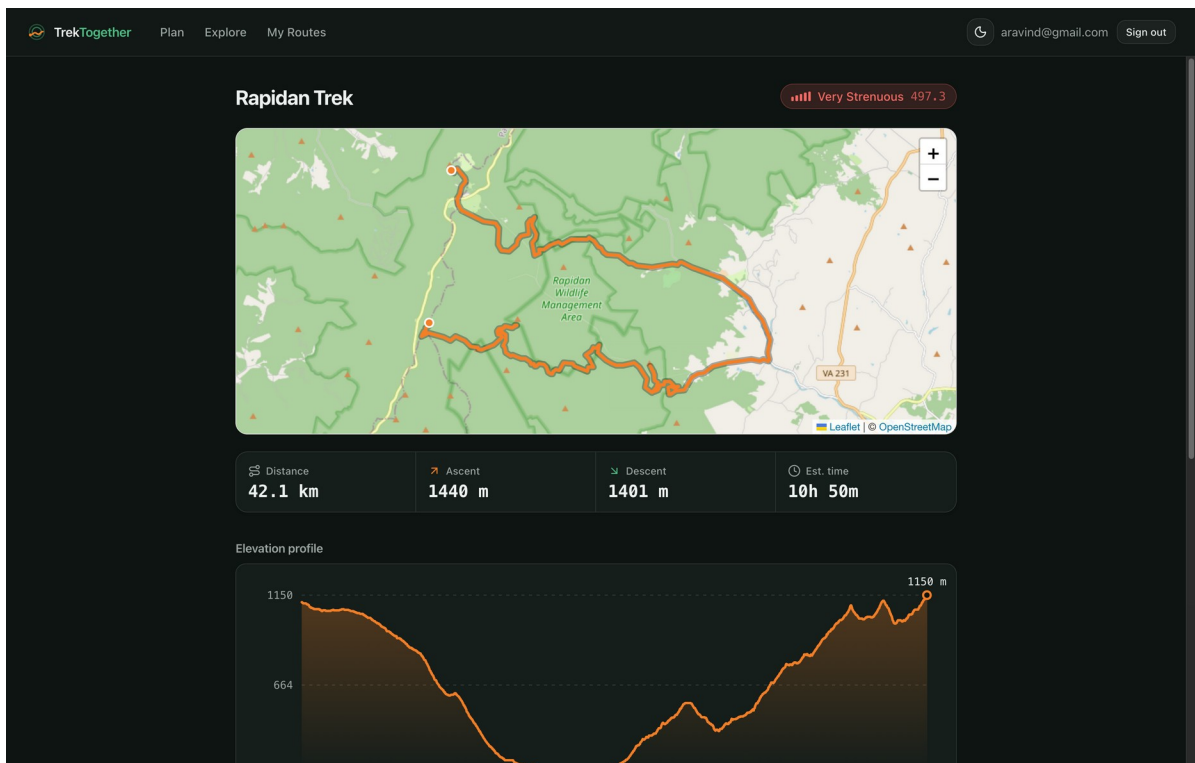


Figure 5.4: Shared route view at /r/:id — the persisted analysis rendered from the share link

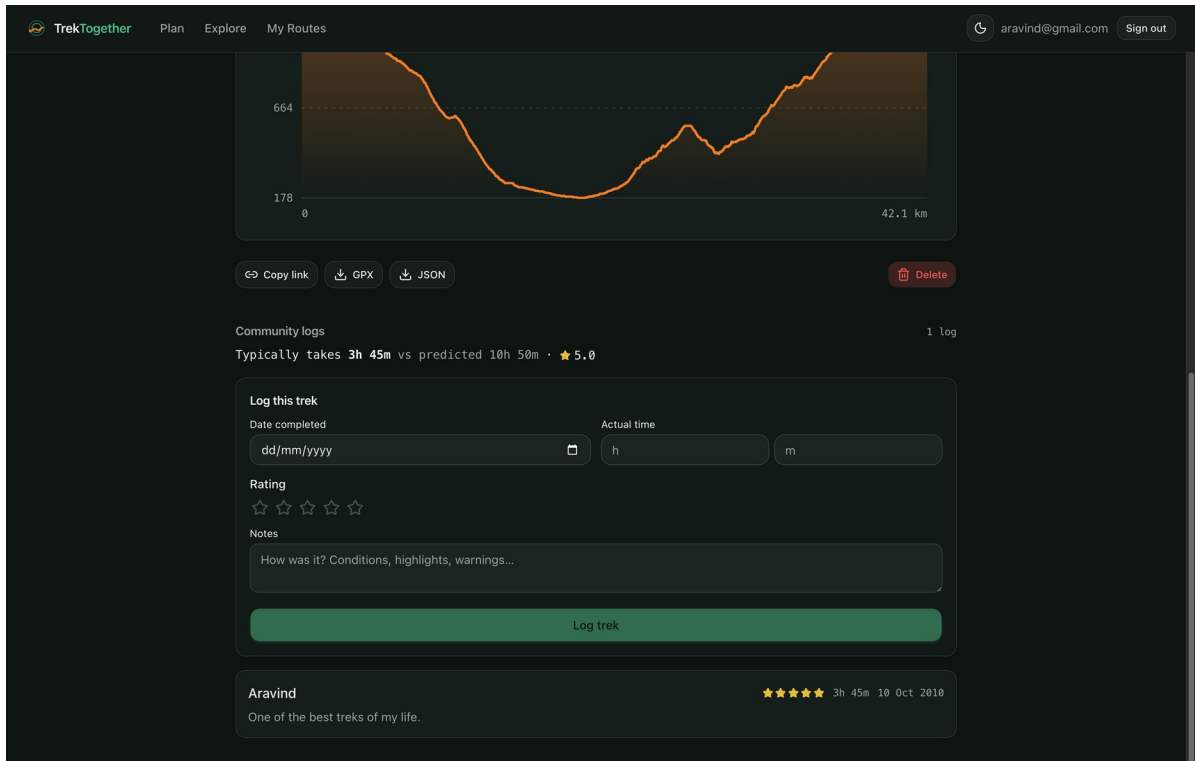


Figure 5.5: Community trek logs — actual duration and rating beside the algorithmic estimates

Deployment Flow

6.1 What is actually built

The project ships “one docker build from production”: a multi-stage server Dockerfile (pnpm workspace install → bundle to a single `dist/index.mjs` → slim Node 24 runtime image), a production `docker-compose.yml` that brings up the API next to a MongoDB 8.2 single-node replica set, and an operations runbook covering environment, boot order, health checks, quota limits, backup, and the hardening that is deliberately documented as not yet implemented (MongoDB keyfile auth, TLS termination). The image was verified end-to-end: the container boots, `/health` reports database connectivity, a real `$geoNear` query succeeds, and the process shuts down gracefully on SIGTERM. The web app builds to static files (plus the service worker) servable by any static host or CDN.

Boot order is load-bearing and scripted: (1) MongoDB healthy with the replica set initiated — the compose healthcheck self-initiates `rs0`; (2) schema + indexes — `db` push followed by the out-of-band index script, because push silently drops the 2dsphere and TTL indexes it does not manage (see the database schema section); (3) the server, which fails fast on invalid environment (Zod-validated at startup) and warns loudly at boot if either unmanaged index is missing.

6.2 Target AWS topology

The intended hosted shape, kept deliberately boring and horizontal-scaling-friendly, is shown in Figure 6.1.

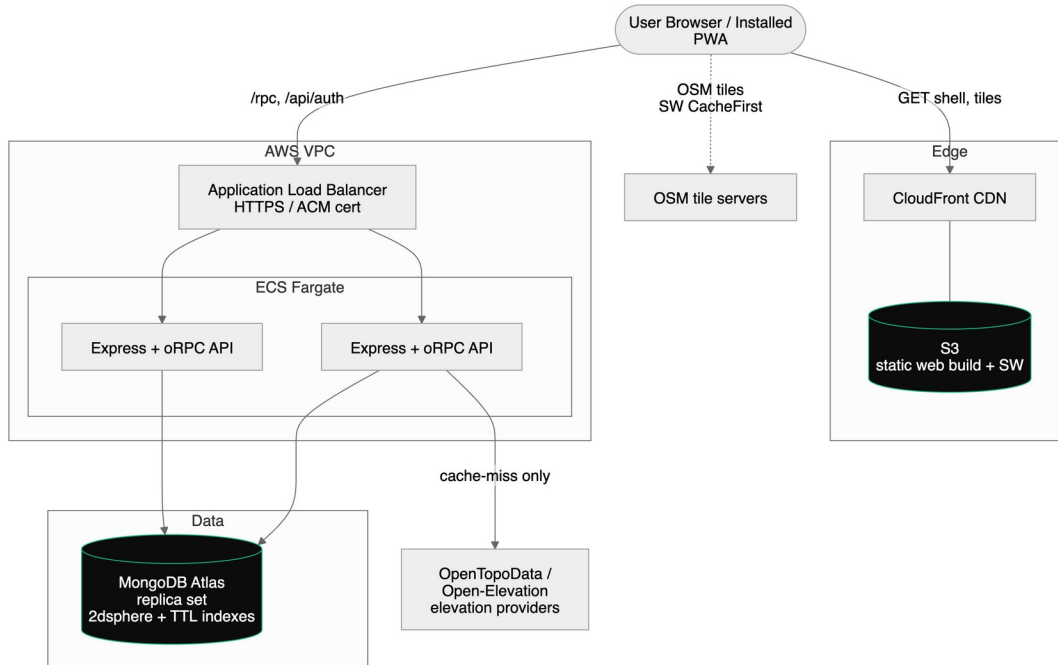


Figure 6.1: Target AWS deployment — CloudFront + S3 edge, ALB + ECS Fargate API inside a VPC, MongoDB Atlas

- **Edge — CloudFront + S3.** The React build and service worker are static artifacts on S3 behind CloudFront. The PWA amplifies the CDN: after first load, the app shell and visited map tiles are cached on-device, so repeat visits barely touch the edge at all.
- **VPC and security groups.** The API runs in private subnets of a VPC. The load balancer's security group admits HTTPS (443) from the internet; the API tasks' security group admits traffic only from the ALB's group; the database accepts connections only from the API tasks (Atlas IP access list / VPC peering). Nothing but the ALB is internet-facing.
- **Application Load Balancer.** Terminates TLS with an ACM certificate, health-checks `/health` (which verifies database connectivity, not just process liveness), and spreads traffic across API tasks.
- **Compute — ECS Fargate.** The server image runs as stateless Fargate tasks. Sessions are cookie + database backed (Better-Auth), so any task can serve any request and horizontal scaling is a task-count change. AWS Elastic Beanstalk on EC2 is the managed-infrastructure alternative: the same image and environment variables apply; the project chose the container path because the artifact is already an image. `NODE_ENV=production` enables Express "trust proxy" so client IPs survive the ALB hop.
- **Database — MongoDB Atlas.** The RDS-equivalent for this stack. The application needs a replica set (Prisma transactions), the 2dsphere geospatial index, and TTL indexes; Atlas provides all three managed, with backups and monitoring. Amazon DocumentDB was evaluated and rejected: its MongoDB compatibility layer does not cover everything the app relies on, while Atlas runs genuine MongoDB. Development mirrors production topology with its single-node `rs0` in Docker.

- **Cache tier.** The elevation cache is deliberately a MongoDB collection, not a separate cache service — it needs zero extra infrastructure, survives restarts, is shared across API tasks by construction, and TTL-expires itself. ElastiCache (Redis) is the documented upgrade path if sub-millisecond cache reads or cross-service reuse ever matter; the data layer's two-method cache repository interface (`findByKeys/upsertMany`) keeps that swap local to one file.
- **External providers.** OpenTopoData/Open-Elevation (elevation), Mapbox (snap-to-trail), Nominatim (geocoding) and OSM tile servers remain third-party HTTP dependencies, reached egress-only — the elevation cache and quota breaker are what make that dependency safe at this tier.

6.3 Environment and secrets

Configuration is twelve-factor: everything arrives as environment variables, validated by a Zod schema at boot so a misconfigured task dies immediately rather than limping. `DATABASE_URL`, `BETTER_AUTH_SECRET` (≥ 32 chars), `BETTER_AUTH_URL` (its scheme drives Secure/SameSite cookie behaviour, fail-secure), `CORS_ORIGIN` (credentials allowed only from the web origin), the optional provider keys, and the quota ceiling. Committed `.env.example` files mirror every key; real secrets never enter the repository — in AWS they would live in Secrets Manager / SSM Parameter Store and be injected into task definitions.

6.4 Delivery pipeline

GitHub Actions runs lint, type-check, tests, and build on every push. The deploy extension is conventional: build and push the server image to ECR, update the ECS service (rolling replacement behind the ALB's health check), and sync the web build to S3 with a CloudFront invalidation. Rollback is redeploying the previous image tag; the database needs no coordinated migration step because schema sync is additive (`db push`) and the index script is idempotent.

Technologies Used

Every technology below is pinned in the repository (Table 7.1) and was chosen for a specific reason; each subsection explains what the technology is, how this project uses it, and where the same tool shows up in industry.

Table 7.1: Core stack and pinned versions

Layer	Technology	Version
Language / runtime	TypeScript (strict) on Node.js	Node 24.17.0
HTTP server	Express	5.1
API layer	oRPC (typed procedures + OpenAPI)	1.13
Validation	Zod	4
Authentication	Better-Auth (email/password sessions)	1.6
Database	MongoDB (replica set)	8.2
ORM	Prisma (mongodb provider)	6.19
Frontend	React + TanStack Router + Vite (PWA)	React 19
Map	Leaflet + OpenStreetMap tiles	react-leaflet
Monorepo	pnpm workspaces + Turborepo	pnpm 11.9
Quality	Biome (lint/format) + Vitest + Husky	Vitest 4
Packaging	Docker multi-stage image	Node 24 base

7.1 Node.js + TypeScript (strict)

Node.js runs JavaScript server-side on an event loop that suits I/O-bound workloads — exactly this app's profile (database reads, third-party HTTP). TypeScript adds static types over it; in strict mode with ES modules it catches whole bug classes (nullability, unit confusion, shape drift) at compile time. Here, one language spans client, server and shared packages, which is what makes end-to-end type inference possible. In industry the pairing is ubiquitous — Netflix, Stripe and Slack all run large TypeScript/Node services.

7.2 Express 5

Express is the minimal, middleware-based HTTP framework that most of the Node ecosystem builds on. This project uses it as a deliberately thin shell: CORS, Better-Auth

mounting, the oRPC handler, a health endpoint, request-id + JSON access logging, graceful shutdown, and centralized error handling. "Top-notch Express" was an explicit grading goal — the discipline is visible less in routes (oRPC owns those) and more in the operational middleware. Express powers production APIs at companies from Uber to IBM; version 5 brings promise-aware routing so async handler errors propagate correctly.

7.3 oRPC + Zod

oRPC provides typed remote procedures over HTTP: the server defines procedures with Zod input/output schemas, and the client *infers* its entire API surface from the server's router type — calls are compile-time checked, and there is no hand-written contract to drift. It is OpenAPI-compatible, so the same procedures also export a spec (checked into docs/api/) and serve interactive reference documentation. Zod is the runtime half: schema-first validation at every trust boundary, from request payloads to environment variables. The pattern — typed RPC with schema validation — is the same one tRPC popularised across the TypeScript industry and GraphQL solves in polyglot settings; oRPC was chosen over tRPC for its first-class OpenAPI story.

7.4 Better-Auth

Better-Auth is a TypeScript-native authentication library: email/password with salted hashing, database-backed sessions, cookie management with environment-aware Secure/SameSite behaviour, all through a Prisma adapter into the same MongoDB. Using a maintained library instead of hand-rolled JWT handling removes an entire category of security mistakes (token invalidation, timing attacks, cookie scoping) — mirroring the industry norm of delegating auth to Auth0/Clerk/Cognito or a hardened library.

7.5 MongoDB + geospatial indexes

MongoDB is a document database: JSON-like documents, flexible per-collection schemas, and — decisive for this project — first-class geospatial support. Geometry is stored as GeoJSON, indexed with **2dsphere** (an index over geometry on the WGS84 sphere), and queried with `$geoNear`, which returns documents ordered by true spherical distance. The elevation cache also leans on Mongo-native features: unique indexes for idempotent

upserts and TTL indexes for automatic expiry. The same geospatial machinery powers “near me” features at Uber-scale ride matching, food delivery, and store locators; the feature-development section's benchmark shows exactly why the index matters.

7.6 Prisma

Prisma is a schema-first ORM: models declared in `.prisma` files generate a fully typed client, so every query result is compile-time checked against the schema. On MongoDB it has two sharp edges this project documents and engineers around (no migrations — db push only; no geo types — raw commands via `aggregateRaw/$runCommandRaw`), which is itself a realistic industry lesson: ORMs buy type safety and productivity, and you keep an escape hatch to the native query language for the 5% they cannot express. Prisma is used in production by companies like Formula 1 teams and fintechs for exactly this typed-data-access role.

7.7 React 19 + TanStack Router + Vite (PWA)

The frontend is a single-page app in React 19 with TanStack Router, whose file-based, fully type-safe routing means even URL params are compile-checked. Vite provides the build tooling; `vite-plugin-pwa` (Workbox under the hood) turns the build into an installable progressive web app with a generated service worker: the app shell precached, OSM tiles in a bounded CacheFirst runtime cache, and the query cache persisted to `localStorage` — which together let a previously viewed route render fully offline, maps included (verified with the API server killed). React Query (TanStack Query) manages server state with request caching and offline persistence. PWAs are the standard answer when a web product needs app-like install and offline behaviour without native builds — Twitter/X Lite and Starbucks are canonical examples.

7.8 Leaflet + OpenStreetMap

Leaflet is the lightweight open-source map library; OpenStreetMap supplies free, community-maintained tiles. Together they cover the two map surfaces (planner and explore) with no API key and no vendor lock-in, with attribution and tile-usage policy honoured (bounded client caching, proper attribution). Leaflet's plugin-free core handles

polylines, markers and click-to-latlng, which is all the planner needs; the same stack underlies countless production mapping features at news organisations, logistics dashboards, and government portals.

7.9 pnpm + Turborepo (monorepo)

The repository is a pnpm workspace: two apps and six packages (`api`, `auth`, `db`, `env`, `ui`, `config`) with strict, content-addressed dependency installs and version pinning via a workspace catalog. Turborepo orchestrates tasks across packages with dependency-aware scheduling and caching — `turbo build` builds only what changed. This is the same shape (at small scale) as the monorepos run by Vercel, Google and Meta: shared types flow between server and client through a package import (`@trek-together/api`), not a published artifact.

7.10 Quality tooling — Biome, Vitest, Husky, GitHub Actions

Biome supplies lint + format + import-sorting in one fast tool (a Rust-based ESLint/Prettier replacement); Vitest runs the 194-test suite; Husky's pre-commit hook runs the linter on staged files so main never receives unformatted code; GitHub Actions repeats lint, type-check, test and build on every push. The pipeline is small but complete — the same gate structure any production TypeScript service runs.

7.11 Docker

Docker packages the server as a reproducible image (multi-stage: workspace install → bundle → slim runtime) and provides the development database (single-node replica set with a self-initiating healthcheck) and the production compose topology. Containers are the deployment lingua franca — the image built here is exactly what ECS Fargate, Kubernetes, or any container platform would run.

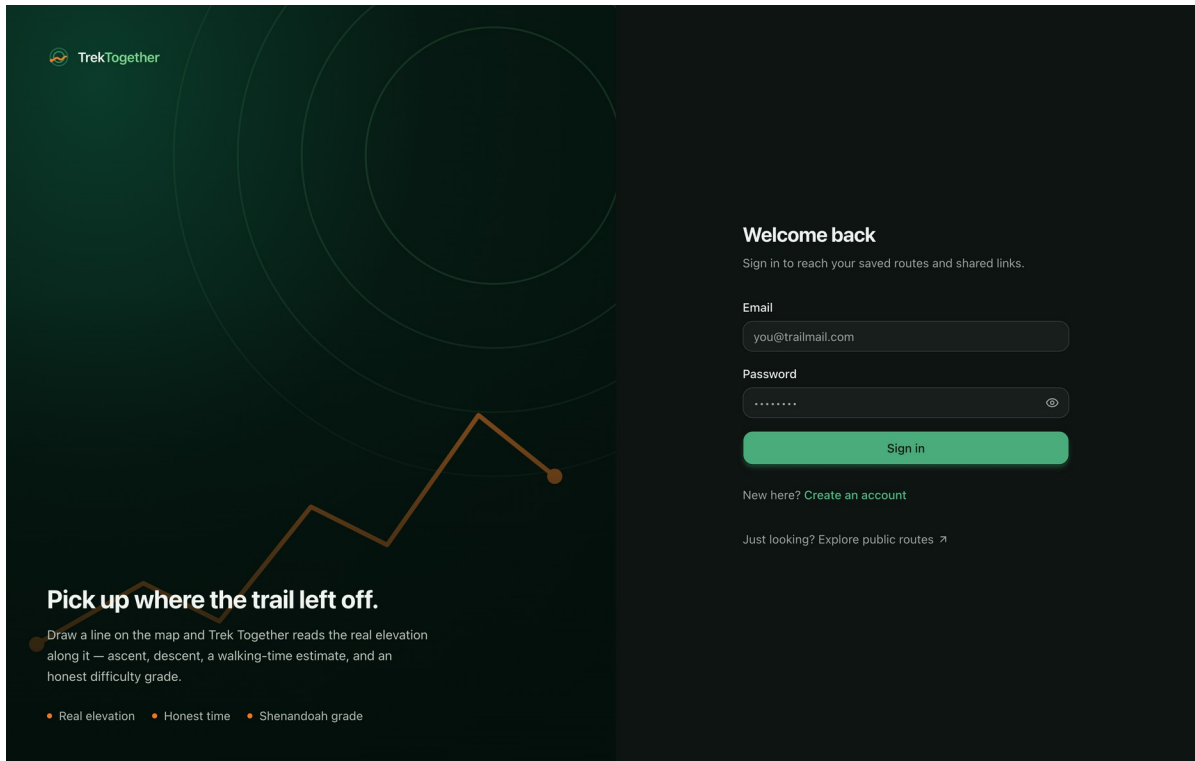


Figure 7.1: Authentication UI (Better-Auth email/password sessions) — the split-screen sign-in screen

Conclusion

8.1 Key takeaways

The project set out to demonstrate backend depth in three specific places — geospatial schema design, an expensive-integration cache, and a typed API flow — and to measure rather than assert its optimisation story. The concepts and skills it consolidated:

- **Document-database design is about access patterns, not normal forms.** Embedding the elevation profile (read-with, bounded, never queried alone) while referencing owners and logs (independent lifecycles, unbounded) made every route view a single document read; the reasoning matters more than the rule.
- **ORMs need escape hatches, and you must know where yours are.** Prisma's type safety carried 95% of data access, but the 2dsphere/TTL indexes and the \$geoNear aggregation live in raw MongoDB commands — and `db.push` will silently drop indexes it does not manage, which the boot-time verifier now guards.
- **Caching rate-limited third parties changes both latency and viability.** The measured $\sim 241\times$ cold-to-warm improvement is the headline, but the deeper lesson is capacity: a free-tier provider at $\sim 1,000$ calls/day supports a real product only if repeat traffic never reaches it. Quantisation, cached nulls, write-through, TTL expiry, a global rate limiter and a daily circuit breaker are each small; together they are the difference between a demo and a service.
- **Types can cross the wire.** With `oRPC` + `Zod`, the client's API surface is inferred from the server; error contracts included. A schema change that would break the UI fails the build, not the demo.
- **Operational discipline is a feature.** Typed degradation, request-correlated logs, graceful shutdown, health checks that test the database, environment fail-fast, WCAG 2.2 AA verification, offline behaviour and a runbook were all buildable in capstone scope — and were what turned the final week's hardening milestone from panic into checklist.

8.2 Practical applications

Each core technique maps directly onto industry work: the `$geoNear` + 2dsphere pattern is the backbone of every “near me” feature (ride-hailing, delivery, retail locators); the cache-first integration wrapper is how production systems consume weather, geocoding, and pricing APIs under quota; the persisted-derived-results schema is the standard shape for analytics-heavy reads; and the typed-RPC contract discipline scales to any TypeScript product team. The application itself is genuinely usable by the hiking community that

inspired it — plan, grade, share, log — and its offline itineraries address the real on-trail condition of no connectivity.

8.3 Limitations

The limitations are documented deliberately, with their costs and upgrade paths:

- **Elevation quota ceiling.** The public OpenTopoData host allows ~1,000 calls/day — roughly 100–200 *cold* analyses across all users. At the ceiling, cold analyses degrade to a typed error until the UTC day rolls; cached terrain is unaffected. Self-hosting OpenTopoData (an ops component plus a multi-GB SRTM download) or a paid provider lifts this; both were consciously deferred.
- **SRTM accuracy bounds the numbers.** ~30 m grid resolution with ~16 m (90% LE) absolute vertical accuracy means short, sharp terrain features can be under-represented; the smoothing that removes noise (5-point MA + 3 m threshold) also slightly flattens real micro-relief. The pipeline's parameters are configurable and documented so the trade-off is inspectable.
- **Time estimates are models, not ground truth.** Naismith and Tobler ignore fitness, load, rest, and trail surface. The community-log layer exists partly to surface this gap: per-route average *actual* duration sits beside the prediction.
- **Prisma-on-MongoDB constraints.** No migration history (schema state is db push-reconciled) and geo operations through raw commands — accepted and engineered around, but a real constraint for teams expecting SQL-style migrations.
- **Deployment hardening not yet implemented.** MongoDB keyfile auth and TLS inside the compose topology are documented as open items in the runbook; the single-node replica set is a dev/prod-shape compromise, not HA. Cost-wise, the free-tier data sources cap scale before infrastructure does.
- **Accessibility limitation.** Map waypoint placement is pointer-only; keyboard waypoint placement is tracked as future work (the surrounding controls and the explore list are fully keyboard-accessible).

8.4 Future work

Natural next steps, in rough order of value: self-hosted elevation data to remove the quota ceiling; route thumbnails and OpenGraph previews for richer sharing; revocable/expiring share tokens for unlisted routes; a Redis tier if cache reads ever dominate; keyboard waypoint editing; and richer community statistics (predicted-vs-actual scatter by difficulty band) once log volume exists.

8.5 Closing

The final system is small by product standards and deliberately so — 92 tasks, 92 dated commits, 194 tests, two measured benchmarks, and one graded question answered honestly: *how hard is this hike, really?* The backend that answers it is layered, typed, cached, indexed, documented, and reproducible from a clean checkout with four commands.

References

All external sources consulted or used during the project are listed below. Formulas, provider limits, and library behaviours cited in the body reference these entries.

- [1] OpenTopoData — Elevation API documentation and public-host limits (100 locations/request, 1 call/sec, 1,000 calls/day). <https://www.opentopodata.org/api/> (accessed June–July 2026).
- [2] Open-Elevation — Free elevation API (public host, ~1,000 requests/month). <https://open-elevation.com/> (accessed June–July 2026).
- [3] U.S. National Park Service — “How to Determine Hiking Difficulty” (Shenandoah rating: $\text{difficulty} = \sqrt{2 \times \text{gain}(\text{ft}) \times \text{distance}(\text{mi})}$ and bands). <https://www.nps.gov/shen/planyourvisit/how-to-determine-hiking-difficulty.htm> (accessed June 2026).
- [4] Naismith's rule — walking-time estimation (1 h per 5 km + 1 h per 600 m ascent). https://en.wikipedia.org/wiki/Naismith%27s_rule (accessed June 2026).
- [5] Tobler, W. (1993) — “Three Presentations on Geographical Analysis and Modeling” (Tobler's hiking function, $W = 6 \cdot e^{(-3.5 \cdot |S+0.05|)}$). Summarised at https://en.wikipedia.org/wiki/Tobler%27s_hiking_function (accessed June 2026).
- [6] Rodríguez, E., Morris, C. S., & Belz, J. E. (2006) — “A Global Assessment of the SRTM Performance”, *Photogrammetric Engineering & Remote Sensing* 72(3) (SRTM ~16 m absolute vertical accuracy, 90% LE).
- [7] MongoDB Manual — Geospatial queries, 2dsphere indexes, \$geoNear, TTL indexes. <https://www.mongodb.com/docs/manual/geospatial-queries/> (accessed June–July 2026).
- [8] Prisma Documentation — MongoDB connector (db push, replica-set requirement, raw commands). <https://www.prisma.io/docs/orm/overview/databases/mongodb> (accessed June–July 2026).
- [9] oRPC — Typed procedures and OpenAPI integration. <https://orpc.unnoq.com/> (accessed June–July 2026).
- [10] Better-Auth — Authentication library documentation. <https://www.better-auth.com/> (accessed June–July 2026).
- [11] TanStack Router & TanStack Query — <https://tanstack.com/router>, <https://tanstack.com/query> (accessed June–July 2026).
- [12] Leaflet — Open-source interactive map library. <https://leafletjs.com/> (accessed June–July 2026).
- [13] OpenStreetMap Foundation — Tile usage policy. <https://operations.osmfoundation.org/policies/tiles/> (accessed July 2026).
- [14] Turborepo — Monorepo task orchestration. <https://turborepo.com/docs> (accessed June 2026).

- [15] Better-T-Stack — Full-stack TypeScript scaffold. <https://better-t-stack.dev/> (accessed June 2026).
- [16] Mapbox — Directions/Map Matching APIs (trail snapping provider). <https://docs.mapbox.com/> (accessed July 2026).
- [17] Nominatim (OpenStreetMap) — Geocoding API used for place search. <https://nominatim.org/> (accessed July 2026).
- [18] Zod — TypeScript-first schema validation. <https://zod.dev/> (accessed June–July 2026).
- [19] Express — Web framework for Node.js, v5. <https://expressjs.com/> (accessed June–July 2026).
- [20] Workbox / vite-plugin-pwa — Service-worker tooling for PWAs. <https://vite-pwa-org.netlify.app/> (accessed July 2026).

AI-assistance acknowledgement. In line with the declaration, the author acknowledges the use of Anthropic's Claude as an AI programming and writing assistant during this project — for code review and scaffolding under the author's direction, and for drafting assistance on documentation including this report. All architectural decisions, requirements, and verification of results are the author's own; all generated material was reviewed, tested, and edited by the author. Project documentation and the dated commit history record this workflow transparently.